Kịch bản thuyết trình Nhóm 6: Verifier-Guided Repair

Đề tài: Verifier-Guided Repair, sửa chứng minh Lean 4 có kiểm chứng dưới ràng buộc chi phí và độ trễ

Môn INT4418 Dữ liệu lớn · Giảng viên: TS. Nguyễn Kiều Linh · Học viện Công nghệ Bưu chính Viễn thông

Bộ slide: slides/Slides_Nhom6_VerifierRepair.pptx, 17 slide gồm 15 slide trình bày và 2 slide phụ lục

Người trình bày	Mã học viên	Slide trình bày
Đào Văn Tâm	B25CHHT112	1, 2, 3, 8, 10, 14, 15
Trần Quang Đức Dũng	B25CHHT088	4, 11
Lâm Thành Trung	B25CHHT117	7, 12
Nguyễn Xuân Tùng	B25CHHT119	9
Khamsing OUTHAIHUENG	B25CHHT125	6, 13
Mekdala Nounou	B25CHHT124	5

Cách dùng

Kịch bản khớp từng trang của bộ slide. Mỗi mục ghi người trình bày, thời lượng ước tính và mốc thời gian cộng dồn. Câu chuyển sang slide sau được in nghiêng. [Chữ xám nhỏ trong ngoặc vuông là chú thích hoặc chỉ dẫn, không đọc thành tiếng.]

Thời lượng ước tính theo khoảng 160 tiếng mỗi phút. Tổng thời lượng phần trình bày là 16:30, mục tiêu là 15 phút. Phần demo ở slide 7 chạy song song với lời nói; nếu cần giải thích thêm trên màn hình thì cộng khoảng 1 phút.

Nếu thời gian bị dài, rút các đoạn đánh dấu [có thể rút] ở slide 9, slide 12, slide 14, tiết kiệm khoảng 1:05. Hai slide phụ lục không trình bày, chỉ mở khi được hỏi.

Lời nói ngắn của từng slide đã được ghi vào phần Notes của file .pptx, mở bằng chế độ Presenter View khi trình chiếu. Khi slide thay đổi, chạy lại python slides/build_slides.py rồi python slides/build_talk_script.py; script dừng và báo lỗi nếu số slide hoặc Notes không khớp.

Bảng thời lượng

Slide	Tiêu đề	Người nói	Thời lượng	Mốc
1	Verifier-Guided Repair	Tâm	1:40	1:40

2	Nhóm 6 nhận chứng minh lỗi từ Nhóm 5, trả chứng minh đã kiểm chứng cho Nhóm 7/8	Tâm	1:00	2:40
3	Điểm nổi bật: hệ thống chạy trọn vòng, đạt 6/6 tiêu chí nghiệm thu	Tâm	0:50	3:30
4	Nút thắt nằm ở quy mô, lập lịch và chi phí, không ở một phép tính	Dũng	0:50	4:20
5	Dataset 17.031 lỗi có nhãn lấy từ chính Lean, không viết tay	Nounou	1:10	5:30
6	Mỗi bản sửa phải qua Lean và kiểm toán trước khi được chấp nhận	Khamsing	0:50	6:20
7	Demo: lỗi → phân loại → sửa → Lean kiểm tra → kiểm toán	Trung	1:00	7:20
8	Cùng dữ liệu, phần cứng và seed, chỉ đổi chiến lược	Tâm	2:15	9:35
9	RQ1: thông báo lỗi tăng 22 điểm % và còn rẻ hơn 45%	Tùng	1:25	11:00
10	RQ2: LLM có chẩn đoán, thử 3 lần đạt 65,1%, nằm trên biên Pareto	Tâm	1:00	12:00
11	RQ3: REPL nhanh 3–4 bậc; 4 worker chỉ nhanh 1,88× vì nghẽn CPU	Dũng	1:15	13:15
12	Phân tích: luật đúng gần hoàn hảo, luật sai kéo chiến lược lai xuống	Trung	1:15	14:30
13	Không có kiểm toán, tỷ lệ thành công bị thổi phồng	Khamsing	0:35	15:05
14	Kết luận: chẩn đoán, kiểm toán, REPL thường trú; sẵn sàng bàn giao	Tâm	0:55	16:00
15	Cảm ơn cô và các bạn · Hỏi đáp	Tâm	0:30	16:30
16	Phụ lục P1 · Ablation: biên dịch và thành công đã kiểm toán	Tâm	phụ lục	
17	Phụ lục P2 · Độ trễ phục vụ P50 và P95 ở 3 mức dữ liệu	Dũng	phụ lục	

Tóm tắt dễ hiểu, đọc trước khi tập

Nguồn: mục Tóm tắt dễ hiểu trong README.md; sửa ở README thì kịch bản cập nhật theo.

Đề tài nói về điều gì

Lean là ngôn ngữ để viết chứng minh toán học sao cho máy tính kiểm tra được từng bước. Một chương trình gọi là trình kiểm chứng sẽ đọc chứng minh và chỉ chấp nhận khi mọi bước đều đúng. Hiện nay các mô hình ngôn ngữ lớn được dùng để viết hàng loạt chứng minh như vậy, nhưng phần lớn chứng minh chúng viết ra bị trình kiểm chứng từ chối. Ví dụ, một chứng minh rằng bình phương của mọi số thực đều không âm có thể bị từ chối chỉ vì thiếu một dòng khai báo thư viện ở đầu tệp. Đề tài xây dựng hệ thống tự động sửa các chứng minh bị từ chối, với điều kiện chỉ nhận bản sửa mà trình kiểm chứng chấp nhận và đề bài không bị thay đổi, đồng thời giữ chi phí và thời gian chờ trong giới hạn.

Nhóm đã làm gì

Nhóm tạo một tập dữ liệu gồm 17.031 chứng minh bị lỗi, mỗi lỗi được gắn nhãn từ chính thông báo của trình kiểm chứng. Hệ thống đọc thông báo lỗi, xếp lỗi vào một trong bảy nhóm, rồi sửa bằng luật viết sẵn, không tốn chi phí, hoặc nhờ mô hình ngôn ngữ lớn sửa và gửi kèm thông báo lỗi. Mỗi bản sửa được trình kiểm chứng kiểm tra lại và được một bộ kiểm toán xác nhận đề bài giữ nguyên. Để kiểm tra nhanh, nhóm giữ trình kiểm chứng chạy liên tục thay vì khởi động lại cho từng chứng minh, và chạy nhiều bản song song. Nhóm đo chất lượng và chi phí ở ba quy mô dữ liệu: 100, 1.000 và 10.000 chứng minh.

Kết quả

Gửi kèm thông báo lỗi cho mô hình nâng tỷ lệ sửa thành công từ 31,7% lên 53,9% và giảm 45% chi phí so với cách chỉ yêu cầu mô hình thử lại. Cho phép thử tối đa ba lần, tỷ lệ thành công đạt 65,1%. Giữ trình kiểm chứng chạy liên tục làm thời gian kiểm tra một chứng minh giảm từ 63 đến 107 giây xuống còn khoảng 3 mili giây. Trong các chứng minh sửa thất bại, 17% đến 26% bản sửa cuối của mô hình đã tự ý thay đổi đề bài và bị bộ kiểm toán chặn. Toàn bộ thực nghiệm tốn 2,97 USD phí gọi mô hình.

Lời trình bày theo từng slide

Slide 1. Verifier-Guided Repair

Người trình bày: Đào Văn Tâm · Thời lượng 1:40 · Mốc 1:40

Em chào cô và các bạn. Em là Đào Văn Tâm, đại diện Nhóm 6. Đề tài của nhóm em là Verifier-Guided Repair, nghĩa là sửa chứng minh toán học có kiểm chứng, trong giới hạn chi phí và độ trễ.

Em xin tóm tắt đề tài trong một phút. Lean [đọc "lin"; ngôn ngữ viết chứng minh toán học và trình kiểm chứng đi kèm] là ngôn ngữ viết chứng minh toán học để máy tính kiểm tra được từng bước, đi kèm thư viện toán học Mathlib [đọc "mát-líp"; viết tắt của Mathematics library; thư viện toán học chuẩn của Lean]. Mô hình ngôn ngữ lớn, gọi tắt là LLM [đọc "eo-eo-em"; viết tắt của Large Language Model; mô hình ngôn ngữ lớn], hiện viết được hàng loạt chứng minh Lean, nhưng phần lớn bị trình kiểm chứng từ chối. Ví dụ, chứng minh rằng bình phương của mọi số thực đều không âm có thể bị từ chối chỉ vì thiếu một dòng khai báo thư viện. Nhóm em xây dựng hệ thống tự động sửa các chứng minh đó bằng luật hoặc bằng LLM, và chỉ nhận bản sửa khi trình kiểm chứng chấp nhận và đề bài giữ nguyên. Kết quả chính: gửi kèm thông báo lỗi nâng tỷ lệ sửa thành công từ 31,7% lên 53,9% và giảm 45% chi phí; giữ trình kiểm chứng chạy liên tục làm thời gian kiểm tra một chứng minh giảm từ hơn một phút xuống khoảng 3 mili giây.

Bài trình bày gồm bốn phần: bài toán; dữ liệu và kiến trúc kèm demo; kết quả của ba câu hỏi nghiên cứu; và kết luận.

Trước hết, em xin trình bày vị trí của Nhóm 6 trong chuỗi dự án của lớp.

Slide 2. Nhóm 6 nhận chứng minh lỗi từ Nhóm 5, trả chứng minh đã kiểm chứng cho Nhóm 7/8

Người trình bày: Đào Văn Tâm · Thời lượng 1:00 · Mốc 2:40

Trong chuỗi dự án AI4Math [đọc "ây-ai pho mát"; viết tắt của Artificial Intelligence for Mathematics; trí tuệ nhân tạo cho toán học, tên chuỗi dự án của lớp] của lớp, Nhóm 5 sinh chứng minh Lean, Nhóm 6 sửa các chứng minh lỗi, rồi chuyển chứng minh đúng cho Nhóm 7 và Nhóm 8.

Mục tiêu gồm ba điểm: chỉ nhận bản sửa mà Lean chấp nhận và đề bài giữ nguyên; chi phí có trần cứng, và độ trễ được đo ở mức thời gian mà 95% số mẫu xử lý xong, gọi là P95 [đọc "pê chín mươi lăm"; viết tắt của Percentile 95; 95% số mẫu xử lý xong trong mức thời gian này]; kết quả tái lập và bàn giao được.

Ba câu hỏi nghiên cứu là: RQ1 [đọc "a-rờ-quy"; viết tắt của Research Question; câu hỏi nghiên cứu], thông báo lỗi của Lean có giúp sửa tốt hơn so với thử lại hay không; RQ2, nên dùng luật hay LLM, thử mấy lần và tốn bao nhiêu; RQ3, hệ thống mở rộng thế nào khi tăng dữ liệu và số tiến trình chạy song song.

Tiếp theo là những gì nhóm đã xây dựng.

Slide 3. Điểm nổi bật: hệ thống chạy trọn vòng, đạt 6/6 tiêu chí nghiệm thu

Người trình bày: Đào Văn Tâm · Thời lượng 0:50 · Mốc 3:30

Nhóm xây dựng bốn thành phần: tập 17.031 chứng minh lỗi có nhãn lấy từ chính Lean; chế độ giữ Lean chạy liên tục, gọi là REPL [đọc "rép-pồ"; viết tắt của Read–Eval–Print Loop; chế độ chạy liên tục: nhận lệnh, thực thi, trả kết quả rồi chờ lệnh tiếp], chạy song song trên nhiều tiến trình, mỗi tiến trình gọi là một worker [đọc "uốc-cơ"; một tiến trình xử lý chạy song song với các tiến trình khác]; vòng sửa gồm luật, LLM và bộ kiểm toán; và cổng nhận dữ liệu từ Nhóm 5 cùng đầu ra cho Nhóm 7 và Nhóm 8.

Bên phải là sáu tiêu chí nghiệm thu tối thiểu của đề bài, và nhóm đạt cả sáu. Toàn bộ thực nghiệm tốn 2,97 USD [đọc "đô la Mỹ"; viết tắt của United States Dollar; đô la Mỹ] phí API [đọc "ây-pi-ai"; viết tắt của Application Programming Interface; giao diện để chương trình gọi một dịch vụ, ở đây là dịch vụ LLM], có 121 bài kiểm thử và 67 bản ghi lần chạy, gọi là run record [đọc "răn ré-cọt"; bản ghi không sửa đổi của một lần chạy, gồm commit, dữ liệu, máy, tham số và chỉ số].

Vì sao đây là bài toán dữ liệu lớn, bạn Dũng xin giải thích.

Slide 4. Nút thắt nằm ở quy mô, lập lịch và chi phí, không ở một phép tính

Người trình bày: Trần Quang Đức Dũng · Thời lượng 0:50 · Mốc 4:20

Em là Trần Quang Đức Dũng. Nút thắt của bài toán nằm ở quy mô, lập lịch và chi phí. Kiểm chứng một tệp theo cách thông thường, tức là khởi động Lean và nạp lại Mathlib, mất 63 đến 107 giây. Ở mức lớn nhất có 10.000 ứng viên, nên kiểm chứng tuần tự cần hơn 7 ngày. Mỗi lần nhờ LLM sửa tốn khoảng 1.000 token [đọc "tô-cần"; đơn vị văn bản mà mô hình xử lý và tính phí]; token là đơn vị văn bản mà mô hình xử lý và tính phí. Vì vậy hệ thống cần xử lý nhiều mẫu mỗi giây, giữ P95 thấp trên vài worker chạy bằng CPU [đọc "xê-pê-u"; viết tắt của Central Processing Unit; bộ xử lý trung tâm], và có trần chi phí.

Tiếp theo, bạn Nounou trình bày về dữ liệu.

Slide 5. Dataset 17.031 lỗi có nhãn lấy từ chính Lean, không viết tay

Người trình bày: Mekdala Nounou · Thời lượng 1:10 · Mốc 5:30

Em là Mekdala Nounou. Dữ liệu gốc là các chứng minh do mô hình Goedel-Prover [đọc "gơ-đồ pru-vơ"; mô hình mã nguồn mở viết chứng minh Lean] viết cho bộ bài toán Lean Workbook [đọc "lin uốc-búc"; bộ bài toán thi đấu đã viết lại bằng Lean], giấy phép MIT [đọc "em-ai-ti"; viết tắt của Massachusetts Institute of Technology; giấy phép mã nguồn mở MIT, cho phép dùng lại tự do]. Lean 4.35 xác nhận 3.666 định lý gốc đúng. Từ đó nhóm tạo lỗi bằng chín cách, như xoá khai báo thư viện hay gõ sai tên một bổ đề, gọi là lemma [đọc "lem-ma"; bổ đề, định lý phụ dùng trong chứng minh]. Ngoài ra có 1.110 lỗi tự nhiên do Mathlib đã đổi tên bổ đề. Tổng cộng có 17.031 mẫu; nhãn lấy từ thông báo thật của Lean, khớp với loại lỗi dự kiến ở 94,4% số mẫu.

Nhóm chia tập theo định lý gốc, nên không định lý nào nằm ở cả hai phía. Tập kiểm tra để riêng, gọi là hold-out [đọc "hâu-ao"; tập kiểm tra để riêng, không dùng khi thiết kế hệ thống], có 284 mẫu ở mức 1.000 và 2.907 mẫu ở mức 10.000. Mỗi tệp dữ liệu có mã kiểm tra toàn vẹn, gọi là checksum [đọc "chéc-xăm"; mã kiểm tra tính toàn vẹn của tệp].

Tiếp theo, bạn Khamsing trình bày kiến trúc hệ thống.

Slide 6. Mỗi bản sửa phải qua Lean và kiểm toán trước khi được chấp nhận

Người trình bày: Khamsing OUTHAIHUENG · Thời lượng 0:50 · Mốc 6:20

Em là Khamsing. Mỗi mẫu đi qua năm bước. Bộ phân tích lỗi, gọi là parser [đọc "pa-xơ"; bộ phân tích thông báo lỗi], xếp lỗi vào một trong bảy nhóm của bảng phân loại, gọi là taxonomy [đọc "tắc-xô-nô-mi"; bảng phân loại lỗi], ví dụ chứng minh để trống bằng lệnh sorry [đọc "so-ri"; lệnh của Lean cho phép bỏ qua một chứng minh], là lệnh bỏ qua chứng minh. Bộ định tuyến, gọi là router [đọc "rao-tơ"; bộ định tuyến chọn cách sửa cho từng lỗi], gửi lỗi đơn giản cho luật viết sẵn, không tốn token, còn lại gửi cho LLM kèm thông báo lỗi. Lean REPL kiểm chứng bản sửa. Bộ kiểm toán yêu cầu không dùng sorry hay axiom [đọc "ắc-xi-ôm"; khai báo một mệnh đề là đúng mà không chứng minh], tức là khai báo một điều đúng mà không chứng minh, và phát biểu định lý giữ nguyên. Vòng lặp có tối đa k [đọc "ca"; số lần thử tối đa cho mỗi mẫu] lần thử, k bằng 1 hoặc 3, và tối đa 300 giây cho mỗi mẫu.

Bạn Trung trình bày phần demo.

Slide 7. Demo: lỗi → phân loại → sửa → Lean kiểm tra → kiểm toán

Người trình bày: Lâm Thành Trung · Thời lượng 1:00 · Mốc 7:20

Em là Lâm Thành Trung. Demo phát lại nhật ký thực nghiệm nên không cần mạng và không tốn chi phí. [Chạy lần lượt ba lệnh trong slides/demo_script.md.]

Mẫu E000880 bị mất dòng import [đọc "im-pót"; dòng khai báo nạp thư viện ở đầu tệp]. Bộ phân loại nhận ra nhóm IMP_02 [đọc "im không hai"; viết tắt của Import, mã nhóm lỗi 02; nhóm lỗi thiếu khai báo thư viện], luật R0 khôi phục dòng khai báo, Lean chấp nhận: 0 token và 73 mili giây.

Mẫu E000003 chỉ dùng LLM. Lần thử thứ nhất biên dịch được nhưng có chứa sorry nên bị kiểm toán chặn; lần thứ hai không biên dịch được; lần thứ ba đúng, tổng cộng 4.583 token. Nếu chỉ đo tỷ lệ biên dịch, lần thứ nhất đã bị tính nhầm là thành công.

Mẫu E000197 sai ở lần thứ nhất; lần thứ hai nhận thông báo lỗi mới và sửa đúng, tốn 779 token.

Bạn Tâm trình bày thiết lập thực nghiệm.

Slide 8. Cùng dữ liệu, phần cứng và seed, chỉ đổi chiến lược

Người trình bày: Đào Văn Tâm · Thời lượng 2:15 · Mốc 9:35

Mọi so sánh dùng cùng dữ liệu, cùng phần cứng và cùng giá trị khởi tạo ngẫu nhiên, gọi là seed [đọc "xít"; giá trị khởi tạo bộ sinh số ngẫu nhiên, cố định để chạy lại ra cùng kết quả]; chỉ chiến lược thay đổi. Phương pháp cơ sở, gọi là baseline [đọc "bết-lai"; phương pháp cơ sở dùng để so sánh], là thử lại với nguyên đoạn yêu cầu ban đầu, gọi là prompt [đọc "prôm"; đoạn văn bản yêu cầu gửi cho mô hình]; các chiến lược khác là chỉ dùng luật, LLM kèm thông báo lỗi, và kết hợp. Ma trận thí nghiệm gồm k bằng 0, 1 hoặc 3; ba mức 100, 1.000 và 10.000 mẫu; 1, 2 hoặc 4 worker; mỗi cấu hình chạy 3 lần với seed 42.

Hạ tầng là máy ảo CPU của Kaggle [đọc "ca-gồ"; nền tảng cung cấp máy ảo miễn phí để chạy notebook], gồm 4 vCPU [đọc "vê xê-pê-u"; viết tắt của virtual CPU; nhân xử lý ảo của máy ảo], tức là 4 nhân xử lý ảo, và 31,3 GB [đọc "ghi-ga-bai"; viết tắt của gigabyte; đơn vị dung lượng, khoảng một tỷ byte] RAM [đọc "ram"; viết tắt của Random Access Memory; bộ nhớ trong của máy], chạy Lean 4.35.0. Mô hình là Gemini [đọc "giê-mi-nai"; họ mô hình LLM của Google] 3.5 Flash-Lite [đọc "phờ-lát lai"; phiên bản nhỏ và rẻ nhất trong họ Gemini Flash], nhiệt độ 0, giá 0,30 USD cho một triệu token đầu vào và 2,50 USD cho một triệu token đầu ra; trần chi phí là 8 USD mỗi tháng. Nhóm có 48 lần chạy trên Kaggle, tổng thời gian chạy cộng dồn khoảng 15 giờ; lần chạy dài nhất khoảng 1,8 giờ.

Độ đo chính là tỷ lệ sửa thành công đã kiểm toán, bằng số mẫu có bản sửa được Lean chấp nhận và qua kiểm toán, chia cho số mẫu hold-out. Ví dụ, 185 trên 284 mẫu là 65,1%. Với RQ3, độ đo chính là thông lượng, tức là số mẫu xử lý được mỗi giây.

Nhóm đặt ba giả thuyết. H1: thông báo lỗi làm tăng tỷ lệ thành công so với baseline. H2: thêm lần thử làm tăng cả chất lượng lẫn chi phí, và luật chỉ có lợi với lỗi đơn giản. H3: REPL gỡ được nút thắt kiểm chứng, nhưng trên một máy, thông lượng không tăng tỷ lệ thuận với số worker.

Bạn Tùng trình bày kết quả của RQ1.

Slide 9. RQ1: thông báo lỗi tăng 22 điểm % và còn rẻ hơn 45%

Người trình bày: Nguyễn Xuân Tùng · Thời lượng 1:25 · Mốc 11:00

Em là Nguyễn Xuân Tùng. RQ1 so sánh hai cách trên cùng 284 mẫu, mỗi mẫu một lần thử: baseline chỉ yêu cầu thử lại, còn cách thứ hai gửi kèm thông báo lỗi. Tỷ lệ thành công tăng từ 31,7% lên 53,9%, tức là tăng 22 điểm phần trăm; chi phí cho 1.000 mẫu giảm từ 2,05 xuống 1,13 USD, tức là giảm 45%.

[có thể rút] Chi phí giảm vì khi có thông báo lỗi, mô hình chỉ sửa chỗ hỏng, trung bình 389 token đầu ra thay vì 783, mà token đầu ra đắt gấp 8,3 lần token đầu vào.

Theo nhóm lỗi, thông báo lỗi giúp nhiều nhất ở lỗi cú pháp SYN_01 [đọc "xin không một"; viết tắt của Syntax, mã nhóm lỗi 01; nhóm lỗi cú pháp], từ 47% lên 85%; lỗi vượt giới hạn tính toán TMO_05 [đọc "tê-em-ô không năm"; viết tắt của Timeout, mã nhóm lỗi 05; nhóm lỗi vượt giới hạn tính toán], từ 10% lên 69%; và IMP_02, từ 3% lên 34%. Hai nhóm SEM_06 [đọc "xem không sáu"; viết tắt của Semantic, mã nhóm lỗi 06; nhóm lỗi chứng minh còn để trống bằng sorry] và TAC_07 [đọc "tắc không bảy"; viết tắt của Tactic, mã nhóm lỗi 07; nhóm lỗi bước chứng minh thất bại] hơi thấp hơn baseline, vì cần viết chứng minh mới. Vậy H1 đúng trên tổng thể, nhưng không đúng cho mọi nhóm lỗi.

Bạn Tâm trình bày kết quả của RQ2.

Slide 10. RQ2: LLM có chẩn đoán, thử 3 lần đạt 65,1%, nằm trên biên Pareto

Người trình bày: Đào Văn Tâm · Thời lượng 1:00 · Mốc 12:00

Trục ngang là chi phí LLM cho 1.000 mẫu, trục dọc là tỷ lệ thành công. Đường chấm nối các cấu hình không bị cấu hình nào khác vừa rẻ hơn vừa tốt hơn, gọi là biên Pareto [đọc "pa-rê-tô"; biên Pareto: các cấu hình không bị cấu hình nào khác vừa rẻ hơn vừa tốt hơn]. Cao nhất là LLM kèm thông báo lỗi với 3 lần thử: 65,1% và 2,45 USD cho 1.000 mẫu. Kết hợp luật với LLM đạt 59,5% với chi phí thấp hơn; chỉ dùng luật đạt 23,9% và không tốn tiền. Baseline và Gemini 3.6 Flash nằm dưới biên; Gemini 3.6 Flash chỉ đạt 36,6%.

H2 đúng ở chỗ thêm lần thử làm tăng cả chất lượng lẫn chi phí. Lý do chiến lược kết hợp thua sẽ có ở phần phân tích.

Bạn Dũng trình bày kết quả của RQ3.

Slide 11. RQ3: REPL nhanh 3–4 bậc; 4 worker chỉ nhanh 1,88× vì nghẽn CPU

Người trình bày: Trần Quang Đức Dũng · Thời lượng 1:15 · Mốc 13:15

Khi giữ Lean chạy liên tục và nạp Mathlib một lần cho mỗi worker, thời gian kiểm chứng một định lý còn 3 mili giây ở mức mà một nửa số mẫu nhanh hơn, gọi là P50 [đọc "pê năm mươi"; viết tắt của Percentile 50; trung vị: một nửa số mẫu xử lý nhanh hơn mức này], và 23 mili giây ở P95, so với 63 đến 107 giây trước đây. Kết quả khớp với cách thông thường ở 74 trên 74 lần kiểm tra.

Mức tăng tốc bằng thông lượng nhiều worker chia cho thông lượng 1 worker. Ở mức 10.000 mẫu, thông lượng tăng từ 1,52 lên 2,86 mẫu mỗi giây khi tăng từ 1 lên 4 worker, tức là 1,88 lần; với 2 worker là 1,76 lần. P95 thời gian phục vụ tăng từ 3,1 giây lên 6,5 giây, vì 4 worker dùng chung 4 nhân CPU. Đúng như H3, muốn mở rộng tiếp thì phải chia việc ra nhiều máy.

Bạn Trung trình bày phần phân tích luật sửa.

Slide 12. Phân tích: luật đúng gần hoàn hảo, luật sai kéo chiến lược lai xuống

Người trình bày: Lâm Thành Trung · Thời lượng 1:15 · Mốc 14:30

Độ chính xác của một luật là tỷ lệ lần áp dụng cho bản sửa qua kiểm toán. R0 khôi phục khai báo thư viện đúng 97%; R7 tăng giới hạn tính toán, đo bằng đơn vị heartbeat [đọc "hát-bít"; đơn vị đo lượng tính toán mà Lean cho phép một chứng minh dùng], đúng 100%. Còn R1 chỉ đúng 27%, R2 và R3 đúng 0%, vì chúng được viết cho đoạn mã ngắn. Router gửi 203 trên 284 mẫu sang luật trước nhưng luật chỉ đúng 33%, nên với một lần thử, chiến lược kết hợp thua LLM.

[có thể rút] Khung bên phải là thí nghiệm loại bỏ từng phần của bộ phân loại để đo đóng góp, gọi là ablation [đọc "ơ-blây-sần"; thí nghiệm loại bỏ từng thành phần để đo đóng góp của thành phần đó], chạy lại từ nhật ký. Bộ phân loại đầy đủ nâng độ khớp nhãn từ 72,3% lên 96,3% và giảm 31% token, nhưng thành công của chiến lược kết hợp giảm từ 34,9% xuống 30,3%, vì nhiều mẫu hơn đi vào luật kém. Điểm cần cải thiện là chỉ dùng luật có độ chính xác cao.

Bạn Khamsing trình bày vai trò của bộ kiểm toán.

Slide 13. Không có kiểm toán, tỷ lệ thành công bị thổi phồng

Người trình bày: Khamsing OUTHAIHUENG · Thời lượng 0:35 · Mốc 15:05

Với LLM ba lần thử, 75,0% số mẫu biên dịch được nhưng chỉ 65,1% thành công thật. Trong các mẫu thất bại, 17% đến 26% bản sửa cuối đã tự ý đổi đề bài, và 10% đến 24% chèn sorry hoặc cấu trúc bị cấm. Bộ kiểm toán chặn sorry, admit [đọc "ợt-mít"; lệnh của Lean bỏ qua chứng minh, tương tự sorry], axiom, native_decide [đọc "nây-típ đi-xai"; lệnh dùng mã biên dịch để kiểm tra, bỏ qua một phần kiểm chứng của Lean] và mọi thay đổi ở phát biểu định lý. Nếu không có bước này, kết quả sẽ được báo cáo cao hơn thực tế.

Bạn Tâm trình bày phần kết luận.

Slide 14. Kết luận: chẩn đoán, kiểm toán, REPL thường trú; sẵn sàng bàn giao

Người trình bày: Đào Văn Tâm · Thời lượng 0:55 · Mốc 16:00

Nhóm rút ra ba bài học: thông báo lỗi của Lean vừa tăng tỷ lệ thành công vừa giảm chi phí; kiểm toán ngữ nghĩa là bắt buộc; và trình kiểm chứng cần được giữ chạy liên tục và lập lịch, vì trên một máy hệ thống bị giới hạn bởi CPU.

Về bàn giao, nhóm có cổng nhận dữ liệu từ Nhóm 5, đầu ra định dạng repaired-v1 [đọc "ri-pe-đờ vê một"; viết tắt của repaired version 1; định dạng đầu ra phiên bản 1 của Nhóm 6] cho Nhóm 7 và Nhóm 8, môi trường đóng gói bằng Docker [đọc "đốc-co"; công cụ đóng gói môi trường chạy để chạy lại giống nhau trên máy khác], 121 bài kiểm thử, các notebook [đọc "nốt-búc"; tệp chứa mã và kết quả chạy theo từng ô] Kaggle để chạy lại, và nhật ký phát lại được.

[có thể rút] Hướng phát triển là chạy trên dữ liệu thật của Nhóm 5, thử mô hình chuyên về Lean chạy trên GPU [đọc "giê-pê-u"; viết tắt của Graphics Processing Unit; bộ xử lý đồ hoạ, thường dùng để chạy mô hình], và chia worker ra nhiều máy bằng hàng đợi Redis [đọc "rê-đít"; viết tắt của Remote Dictionary Server; hệ thống lưu dữ liệu trong bộ nhớ, ở đây dùng làm hàng đợi công việc] mà bạn Dũng đã xây dựng.

Cuối cùng là phân công và hỏi đáp.

Slide 15. Cảm ơn cô và các bạn · Hỏi đáp

Người trình bày: Đào Văn Tâm · Thời lượng 0:30 · Mốc 16:30

Em xin cảm ơn cô và các bạn đã lắng nghe. Khi hỏi đáp, câu hỏi về đánh giá và chi phí xin chuyển cho em; về worker, REPL và khả năng mở rộng cho bạn Dũng; về luật sửa cho bạn Trung; về LLM, prompt và việc chọn mô hình cho bạn Tùng; về bộ phân loại và kiểm toán cho bạn Khamsing; về dữ liệu cho bạn Nounou. Nhóm em rất mong nhận được câu hỏi và góp ý của cô và các bạn.

Slide 16. Phụ lục P1 · Ablation: biên dịch và thành công đã kiểm toán

Người trình bày: Đào Văn Tâm · Phụ lục, không trình bày

[Không trình bày. Mở khi được hỏi về tỷ lệ biên dịch so với thành công thật ở từng cấu hình: khoảng chênh giữa cột xanh và cột cam là phần bộ kiểm toán chặn.]

Slide 17. Phụ lục P2 · Độ trễ phục vụ P50 và P95 ở 3 mức dữ liệu

Người trình bày: Trần Quang Đức Dũng · Phụ lục, không trình bày

[Không trình bày. Mở khi được hỏi về P95 ở từng mức dữ liệu: biểu đồ cho P50 và P95 ở 100, 1.000 và 10.000 mẫu với 1, 2 và 4 worker, trung vị và khoảng min đến max của 3 lần lặp.]

Phụ lục A. Số liệu thực nghiệm và hạ tầng

Mục	Giá trị	Nguồn
Máy chạy thực nghiệm	Kaggle CPU, Linux-6.18.48+-x86_64, 4 vCPU,	results/raw/run_exp-

	31,3 GB RAM	*.json, trường machine
Máy phát triển	macOS arm, 8 nhân, 16 GB RAM; chỉ dùng để phát triển và kiểm thử	results/raw, run record chạy trên macOS
Phần mềm	Lean 4.35.0, Mathlib cùng phiên bản, Python 3.13.15	trường software của run record; lean_ws/lake-manifest.json
Huấn luyện mô hình	Không huấn luyện; LLM được gọi qua API	thiết kế hệ thống
Cấu hình LLM	gemini-3.5-flash-lite; temperature 0.0; seed 42; thinking_level minimal; tối đa 2048 token đầu ra	src/llm_client.py
Giá LLM	0,30 USD cho 1 triệu token vào; 2,50 USD cho 1 triệu token ra	configs/llm_prices.yaml
Hạn mức chi phí	8 USD mỗi tháng; 2 USD mỗi lần chạy	configs/budget.yaml
Chi phí thực tế	2,97 USD qua 2028 lần gọi được ghi sổ	results/cost_ledger.jsonl
Thực nghiệm trên Kaggle	48 lần chạy; tổng thời gian chạy cộng dồn 15 giờ, các phiên chạy song song; 3 lần lặp mỗi cấu hình; seed 42	results/raw/run_exp-*.json
Chuẩn bị dữ liệu trên Kaggle	kiểm chứng định lý gốc 1,3 giờ; sinh và gắn nhãn lỗi 0,6 giờ	run_build-seeds, run_gen-errors
Dữ liệu đánh giá	17.031 mẫu lỗi từ 3.666 định lý gốc, gồm 15.921 lỗi tổng hợp và 1.110 lỗi tự nhiên; hold-out 284 mẫu ở mức 1.000 và 2.907 mẫu ở mức 10.000	run_gen-errors; reports/report.md mục 3
Phiên Kaggle	2026-10-17: 51 run record	trường timestamp của run record
Notebook	run2b_llm_PAID.ipynb, run3a_scale_10k.ipynb, run3b_scale_100_1k.ipynb	notebooks/sessions/

Thời gian mỗi lần chạy theo cấu hình, trung vị các lần lặp:

Chiến lược	k	Worker	Dữ liệu	Model	Số lần	Thời gian mỗi lần
rule_only	1	1	100	-	3	2 phút
rule_only	1	2	100	-	3	1,2 phút
rule_only	1	4	100	-	3	1,3 phút
hybrid	1	4	10k	gemini-3.5-flash-lite	1	24,1 phút
rule_only	1	1	10k	-	3	110 phút
rule_only	1	2	10k	-	3	62,5 phút
rule_only	1	4	10k	-	3	58,7 phút
blind	1	4	1k	gemini-3.5-flash-lite	3	3,9 phút
hybrid	1	4	1k	gemini-3.5-flash-lite	3	2,8 phút
hybrid	3	4	1k	gemini-3.5-flash-lite	3	6,2 phút
llm_only	1	4	1k	gemini-3.5-flash-lite	3	4 phút
llm_only	1	4	1k	gemini-3.6-flash	1	7,2 phút
llm_only	3	4	1k	gemini-3.5-flash-lite	3	6 phút

none	0	4	1k	-	2	0,3 phút
rule_only	1	1	1k	-	3	12,9 phút
rule_only	1	2	1k	-	3	7,9 phút
rule_only	1	4	1k	-	3	7,2 phút
rule_only	3	4	1k	-	2	1,8 phút

Phụ lục B. Thuật ngữ, tên riêng, viết tắt

Thuật ngữ	Cách đọc	Viết tắt của	Nghĩa
ablation	ơ-blây-sần	-	thí nghiệm loại bỏ từng thành phần để đo đóng góp của thành phần đó
admit	ợt-mít	-	lệnh của Lean bỏ qua chứng minh, tương tự sorry
AI4Math	ây-ai pho mát	Artificial Intelligence for Mathematics	trí tuệ nhân tạo cho toán học, tên chuỗi dự án của lớp
API	ây-pi-ai	Application Programming Interface	giao diện để chương trình gọi một dịch vụ, ở đây là dịch vụ LLM
axiom	ắc-xi-ôm	-	khai báo một mệnh đề là đúng mà không chứng minh
baseline	bết-lai	-	phương pháp cơ sở dùng để so sánh
checksum	chéc-xăm	-	mã kiểm tra tính toàn vẹn của tệp
CPU	xê-pê-u	Central Processing Unit	bộ xử lý trung tâm
Docker	đốc-cơ	-	công cụ đóng gói môi trường chạy để chạy lại giống nhau trên máy khác
Flash-Lite	phờ-lát lai	-	phiên bản nhỏ và rẻ nhất trong họ Gemini Flash
GB	ghi-ga-bai	gigabyte	đơn vị dung lượng, khoảng một tỷ byte
Gemini	giê-mi-nai	-	họ mô hình LLM của Google
Goedel-Prover	gơ-đồ pru-vơ	-	mô hình mã nguồn mở viết chứng minh Lean
GPU	giê-pê-u	Graphics Processing Unit	bộ xử lý đồ hoạ, thường dùng để chạy mô hình
heartbeat	hát-bít	-	đơn vị đo lượng tính toán mà Lean cho phép một chứng minh dùng
hold-out	hâu-ao	-	tập kiểm tra để riêng, không dùng khi thiết kế hệ thống
IMP_02	im không hai	Import, mã nhóm lỗi 02	nhóm lỗi thiếu khai báo thư viện
import	im-pót	-	dòng khai báo nạp thư viện ở đầu tệp
k	ca	-	số lần thử tối đa cho mỗi mẫu
Kaggle	ca-gồ	-	nền tảng cung cấp máy ảo miễn phí để chạy notebook

Lean	lin	-	ngôn ngữ viết chứng minh toán học và trình kiểm chứng đi kèm
Lean Workbook	lin uốc-búc	-	bộ bài toán thi đấu đã viết lại bằng Lean
lemma	lem-ma	-	bổ đề, định lý phụ dùng trong chứng minh
LLM	eo-eo-em	Large Language Model	mô hình ngôn ngữ lớn
Mathlib	mát-líp	Mathematics library	thư viện toán học chuẩn của Lean
MIT	em-ai-ti	Massachusetts Institute of Technology	giấy phép mã nguồn mở MIT, cho phép dùng lại tự do
native_decide	nây-típ đi-xai	-	lệnh dùng mã biên dịch để kiểm tra, bỏ qua một phần kiểm chứng của Lean
notebook	nốt-búc	-	tệp chứa mã và kết quả chạy theo từng ô
P50	pê năm mươi	Percentile 50	trung vị: một nửa số mẫu xử lý nhanh hơn mức này
P95	pê chín mươi lăm	Percentile 95	95% số mẫu xử lý xong trong mức thời gian này
Pareto	pa-rê-tô	-	biên Pareto: các cấu hình không bị cấu hình nào khác vừa rẻ hơn vừa tốt hơn
parser	pa-xơ	-	bộ phân tích thông báo lỗi
prompt	prôm	-	đoạn văn bản yêu cầu gửi cho mô hình
RAM	ram	Random Access Memory	bộ nhớ trong của máy
Redis	rê-đít	Remote Dictionary Server	hệ thống lưu dữ liệu trong bộ nhớ, ở đây dùng làm hàng đợi công việc
repaired-v1	ri-pe-đờ vê một	repaired version 1	định dạng đầu ra phiên bản 1 của Nhóm 6
REPL	rép-pồ	Read–Eval–Print Loop	chế độ chạy liên tục: nhận lệnh, thực thi, trả kết quả rồi chờ lệnh tiếp
router	rao-tơ	-	bộ định tuyến chọn cách sửa cho từng lỗi
RQ	a-rờ-quy	Research Question	câu hỏi nghiên cứu
run record	răn ré-cọt	-	bản ghi không sửa đổi của một lần chạy, gồm commit, dữ liệu, máy, tham số và chỉ số
seed	xít	-	giá trị khởi tạo bộ sinh số ngẫu nhiên, cố định để chạy lại ra cùng kết quả
SEM_06	xem không sáu	Semantic, mã nhóm lỗi 06	nhóm lỗi chứng minh còn để trống bằng sorry

sorry	so-ri	-	lệnh của Lean cho phép bỏ qua một chứng minh
SYN_01	xin không một	Syntax, mã nhóm lỗi 01	nhóm lỗi cú pháp
TAC_07	tắc không bảy	Tactic, mã nhóm lỗi 07	nhóm lỗi bước chứng minh thất bại
taxonomy	tắc-xô-nô-mi	-	bảng phân loại lỗi
TMO_05	tê-em-ô không năm	Timeout, mã nhóm lỗi 05	nhóm lỗi vượt giới hạn tính toán
token	tô-cần	-	đơn vị văn bản mà mô hình xử lý và tính phí
USD	đô la Mỹ	United States Dollar	đô la Mỹ
vCPU	vê xê-pê-u	virtual CPU	nhân xử lý ảo của máy ảo
worker	uốc-cơ	-	một tiến trình xử lý chạy song song với các tiến trình khác

Phụ lục C. Cách đọc ký hiệu

Ký hiệu	Cách đọc	Nghĩa
k=1, k=3	ca bằng một, ca bằng ba	số lần thử tối đa cho mỗi mẫu
P50, P95	pê năm mươi, pê chín mươi lăm	phân vị 50 và 95 của thời gian xử lý
1K, 10K	một nghìn, mười nghìn	1.000 và 10.000 mẫu
→	lên, sang, hoặc chuyển thành	thay đổi từ giá trị trước sang giá trị sau
×	lần	hệ số nhân, ví dụ 1,88× là 1,88 lần
⊂	là tập con của	100 ⊂ 1K ⊂ 10K: tập nhỏ nằm trong tập lớn
~	khoảng	giá trị xấp xỉ
$	đô la Mỹ	đơn vị tiền, đọc sau con số: $2,45 là hai phẩy bốn lăm đô la Mỹ
%	phần trăm	tỷ lệ trên 100
điểm %	điểm phần trăm	hiệu của hai tỷ lệ phần trăm, ví dụ 53,9% trừ 31,7% là 22 điểm phần trăm
ms, s	mili giây, giây	đơn vị thời gian, 1 giây bằng 1.000 mili giây
·	và	dấu phân cách giữa các mục trên slide
min–max	nhỏ nhất đến lớn nhất	khoảng biến thiên giữa các lần lặp